VLSI Internship – Task 2 Report

Introduction to Verilog HDL and RTL Design of Basic Combinational Circuits

Submitted by

Name : Payal Verma

College : Raj Kumar Goel Institute of Technology

Branch : Electronics and Computer Engineering

Internship : VLSI Internship

Task : Task 2

Table of Contents

1. Introduction

Verilog HDL (Hardware Description Language) is one of the most widely used hardware description languages for designing and verifying digital circuits. It enables engineers to describe the behavior and structure of digital systems before implementing them on hardware such as FPGAs or ASICs.

In this task, various combinational logic circuits including basic logic gates, Half Adder, and Full Adder were designed using Verilog HDL. Each design was verified using a testbench, and simulation waveforms were analyzed to ensure correct functionality.

2. Objective

The objectives of this task are:

- To understand the basics of Verilog HDL.
- To learn RTL design concepts.
- To design basic combinational circuits.
- To write Verilog modules.
- To create testbenches.
- To simulate the circuits.
- To analyze waveform outputs.

3. Software Used

- Xilinx Vivado
- Verilog HDL
- Behavioral Simulation
- Waveform Viewer

4. Logic Gates

4.1 - And Gate

The AND gate produces HIGH output only when both inputs are HIGH.

Boolean Expression

$Y = A \cdot B$

Truth Table

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

Design Code

```
module and_gate(a, b, y);
input a, b;
output y;
assign y = a & b;
endmodule
```

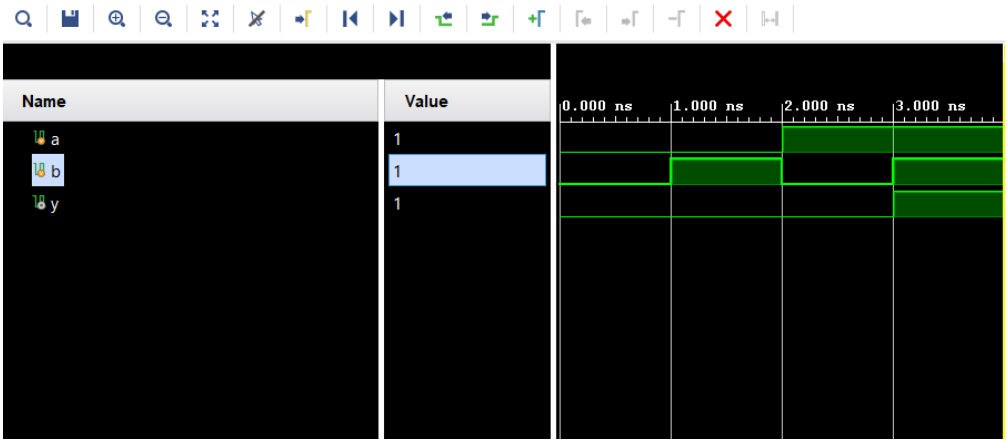
Testbench

```
module tb_and ();
reg a, b;
wire y;
and_gate dut (a, b, y);
initial begin
$monitor("Output is a:%b, b:%b, y:%b", a, b, y);

a=0; b=0; #1;
a=0; b=1; #1;
a=1; b=0; #1;
a=1; b=1; #1;

$finish;
end
endmodule
```

Simulation Waveform



4.2 - Or Gate

The OR gate produces HIGH output when at least one input is HIGH.

Boolean Expression

Y = A + B

Truth Table

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

Design Code

```
module or_gate(a, b,  y);
input a, b;
output y;
assign y = (a | b);
endmodule
```

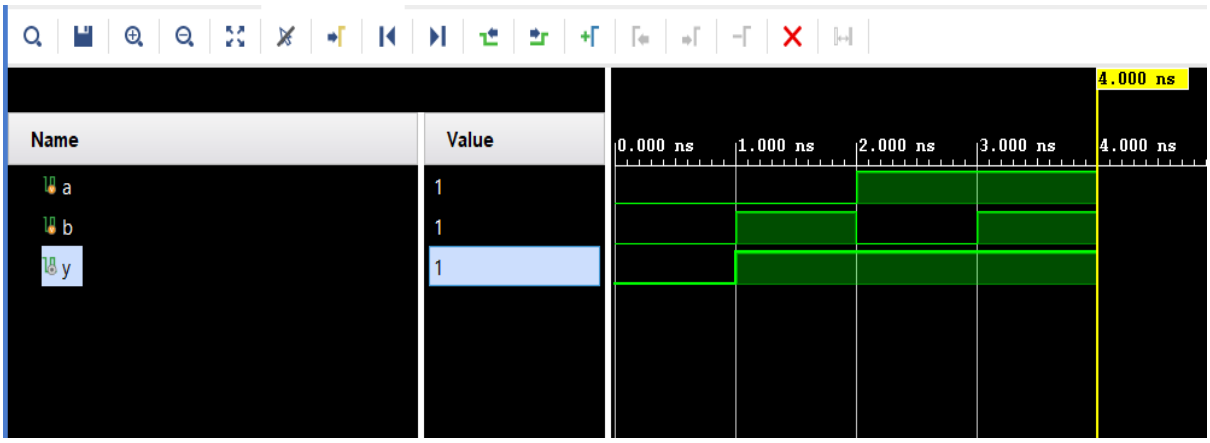
Testbench

```
module tb_or ();
reg a, b;
wire y;
or_gate dut (a,b,y);
initial begin
$monitor("The O/p is a:%b, b:%b, y:%b",a, b, y);

a=0; b=0; #1;
a=0; b=1; #1;
a=1; b=0; #1;
a=1; b=1; #1;

$finish; #1;
end
endmodule
```

Simulation Waveform



4.3- Nor Gate

It produces a HIGH (1) output only when all inputs are LOW (0

Boolean Expression

$Y=\overline{A+B}$

Truth Table

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

Design Code

```
module nor_gate (a, b, y);
input a, b;
output y;
assign y = ~ (a | b);
endmodule
```

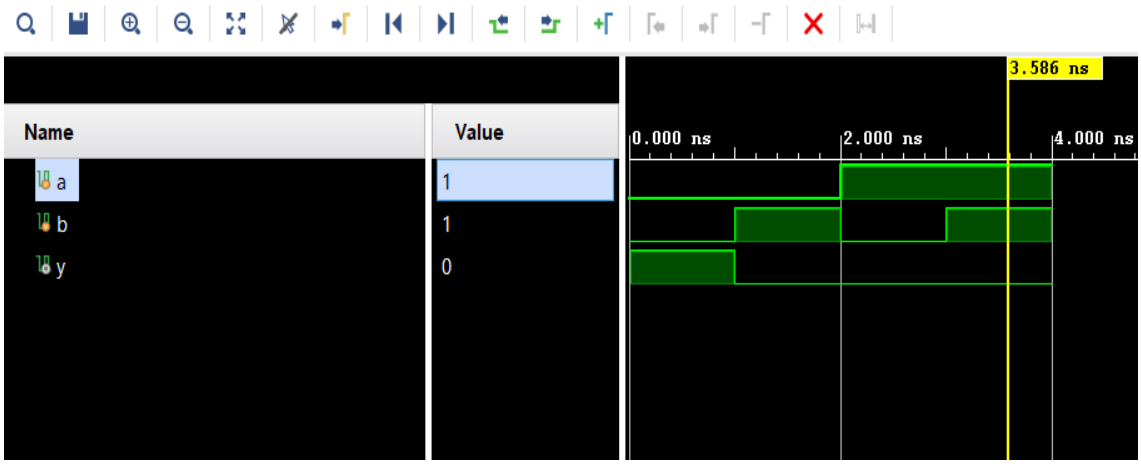
Testbench

```
module tb_nor ();
reg a, b;
wire y;
nor_gate dut (a, b, y);
initial begin
$monitor("The O/p is a:%b, b:%b, y:%b", a, b,  y);

a=0; b=0; #1;
a=0; b=1; #1;
a=1; b=0; #1;
a=1; b=1; #1;

$finish; #1;
end
endmodule
```

Simulation Waveform



4.4 - Nand Gate

The NAND gate is the complement of the AND gate.

Boolean Expression

$Y = (A \cdot B)'$

Truth Table

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

Design Code

```
module nand_gate(a, b, y);
input a, b;
output y;
assign y = ~ (a & b);
endmodule
```

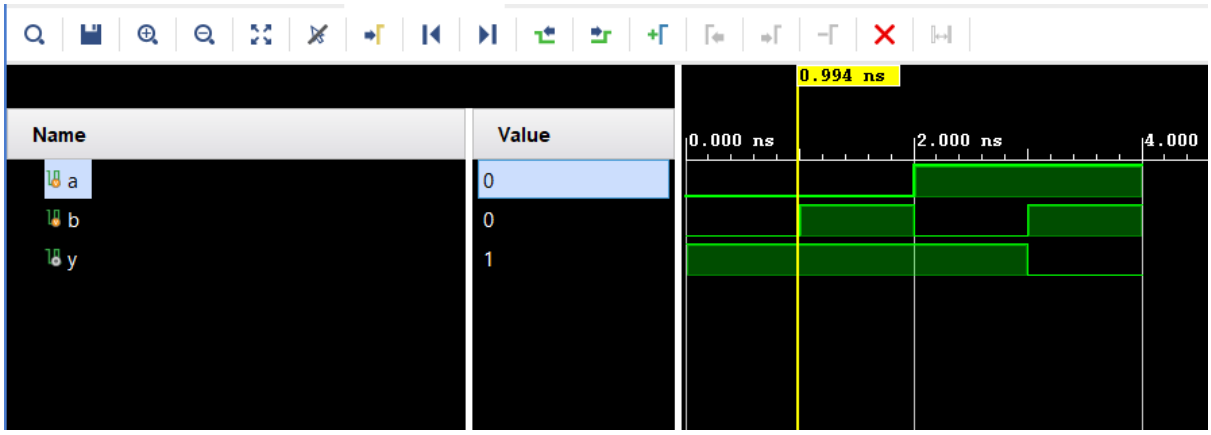
Testbench

```
module tb_nand();
reg a, b;
wire y;
nand_gate dut(a ,b, y);
initial begin
$monitor("The O/p is a:%b, b:%b, y:%b", a, b, y);

a=0; b=0; #1;
a=0; b=1; #1;
a=1; b=0; #1;
a=1; b=1; #1;

$finish; #1;
end
endmodule
```

Simulation Waveform



4.5 - Xor Gate

The XOR gate produces HIGH output when the inputs are different.

Boolean Expression

$Y = A \oplus B$

Truth Table

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

Design Code

```
module xor_gate(a, b, y);
input a, b;
output y;
assign y = (a ^ b);
endmodule
```

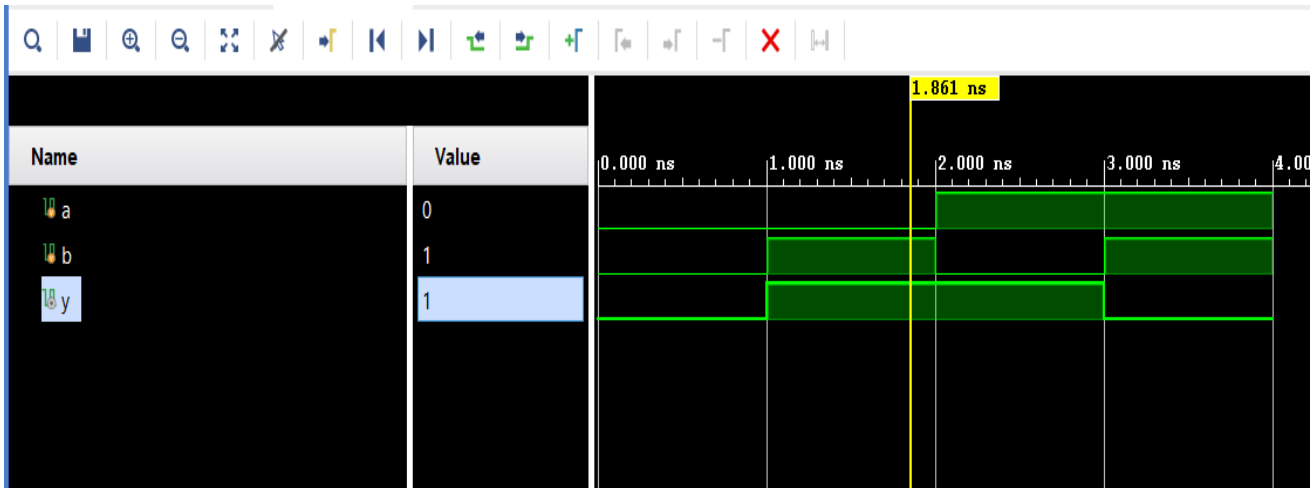
Testbench

```
module tb_xor();
reg a, b;
wire y;
xor_gate dut(a, b, y);
initial begin
$monitor("The O/p is a:%b, b:%b, y:%b",a ,b ,y);

a=0; b=0; #1;
a=0; b=1; #1;
a=1; b=0; #1;
a=1; b=1; #1;

$finish; #1;
end
endmodule
```

Simulation Waveform



5 Half Adder

A Half Adder is a combinational circuit that performs the addition of two one-bit binary numbers.

Boolean Expressions

- $\text{Sum (S)} = A \oplus B$
- $\text{Carry (C)} = A \cdot B$

Truth Table

A	B	Carry	Sum
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Design Code

```
module half_adder(a, b, sum, carry);
input a, b;
output sum, carry;
assign sum = (a ^ b);
assign carry = (a & b);
endmodule
```

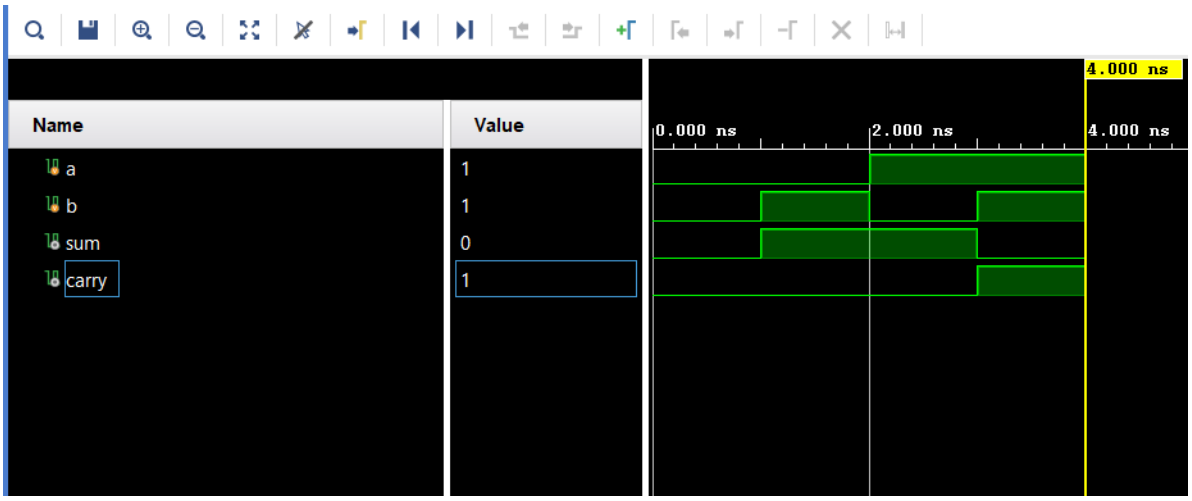
Testbench

```
module tb_ha();
reg a, b;
wire sum, carry;
half_adder dut(a, b, sum, carry);
initial begin
$monitor("a:%b, b:%b, sum:%b, carry:%b",a, b, sum,
carry);

a=0; b=0; #1;
a=0; b=1; #1;
a=1; b=0; #1;
a=1; b=1; #1;

$finish; #1;
end
endmodule
```

Simulation Waveform



6. Full Adder

A Full Adder is a combinational circuit used to add three one-bit binary numbers (A, B, Cin).

Boolean Expressions

- Sum= A ⊕ B ⊕ Cin
- Carry =AB+ (A⊕B) Cin

Truth Table

A	B	Cin	Sum	Carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Design Code

```
module full_adder(a, b, cin, sum,  carry);
input a, b, cin;
output carry, sum;

assign sum = (a ^ b ^ cin);
assign carry = (a &b | b & cin | a & cin);

endmodule
```

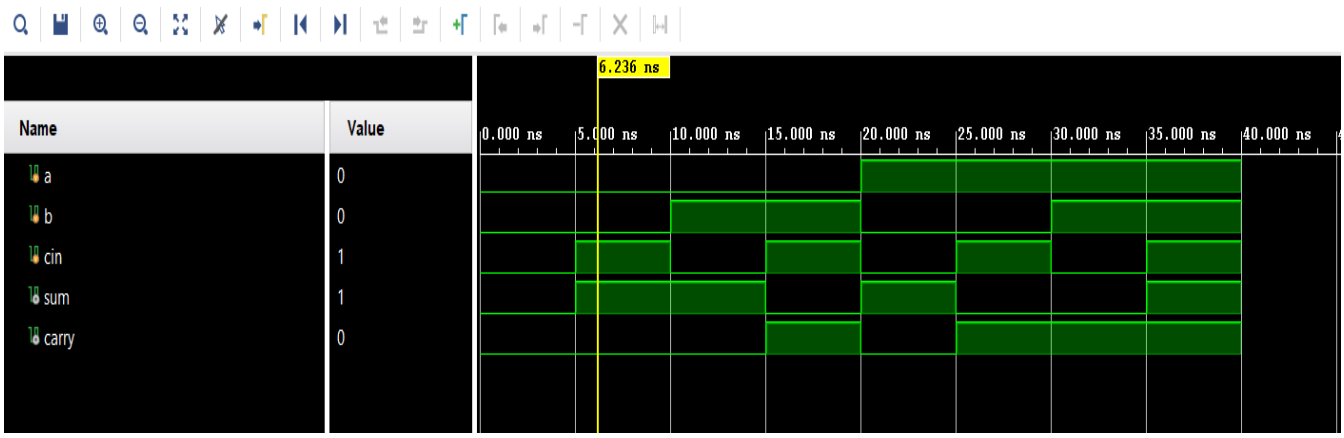
Testbench

```
module tb_full_adder();
reg a, b, cin;
wire sum, carry;
full_adder dut(a, b, cin, sum, carry);
initial begin
$monitor("a:%b, b:%b, cin:%b, sum:%b, carry:%b", a, b, cin,
sum, carry);

a=0; b=0; cin=0; #5;
a=0; b=0; cin=1; #5;
a=0; b=1; cin=0; #5;
a=0; b=1; cin=1; #5;
a=1; b=0; cin=0; #5;
a=1; b=0; cin=1; #5;
a=1; b=1; cin=0; #5;
a=1; b=1; cin=1; #5;

$finish; #5;
 end
endmodule
```

Simulation Waveform



7. Simulation Results

The designed circuits were simulated successfully using the Vivado simulator. Testbenches applied all possible input combinations, and the generated outputs matched the expected truth tables. Waveforms confirmed the correctness of each combinational circuit.

8. Waveform Analysis

The simulation waveform verifies that:

- Input signals change according to the testbench.
- Output changes immediately after input changes.
- No timing errors were observed.
- All logic gates function correctly.
- Half Adder outputs match the expected truth table.
- Full Adder outputs match the expected truth table.

- Simulation successfully verifies the functionality of all implemented circuits

9. Conclusion

In this task, Verilog HDL was used to design and simulate several basic combinational circuits, including logic gates, Half Adder, and Full Adder. Testbenches were developed to verify each module, and simulation waveforms confirmed the correctness of the outputs. This task provided practical experience in RTL design, Verilog coding, testbench development, and waveform analysis, forming a strong foundation for more advanced digital design projects.

10. Learning Outcomes

After completing this task, I learned:

- Basics of Verilog HDL
- RTL Design Concepts
- Module Creation
- Combinational Circuit Design
- Logic Gate Implementation
- Half Adder Design
- Full Adder Design
- Testbench Writing
- Simulation Process
- Waveform Analysis